# TCSS 343 – Design and Analysis of Algorithms

Recursive Search Algorithms

# Recursive Algorithms

- What makes a recursive algorithm different from iterative algorithms?
  - A recursive algorithm solves the simple cases no differently.
  - A recursive algorithm solves the more difficult cases by reducing them to simpler versions of the same problem.

- This technique is called divide-and-conquer.
  - This is our first algorithm design strategy.

- Example Problem: Given input of an unsorted or sorted list $A$ and a target value $t$ output the index of $t$ in $A$ if it exists.

Search an Unsorted List (SAUL)
Input: $A[1 \dots n]$ is a list of integers and $t$ is an integer
Output: $i \in \{1, \dots, n\}$ such that $A[i] = t$ if such exists and $-1$ otherwise.

Search a Sorted List (SASL)
Input: $A[1 \dots n]$ is a list of integers such that $A[1] \leq A[2] \leq \dots \leq A[n]$ and $t$ is an integer
Output: $i \in \{1, \dots, n\}$ such that $A[i] = t$ if such exists and $-1$ otherwise.

# Divide and Conquer Design

- Divide and Conquer Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Design a recursive algorithm from the structure of the self-reduction.
  5. Analyse the runtime of the algorithm using a recurrence.

- Let's apply this design strategy to the simple problem of searching an unsorted list (SAUL).
  - We have already formalized this problem. That's step 1.

Search an Unsorted List (SAUL)
Input: $A[1 \ldots n]$ is a list of integers and $t$ is an integer
Output: $i \in \{1, \ldots, n\}$ such that $A[i] = t$ if such exists and $-1$ otherwise.

- To find a self-reduction we must formally express the output of the main problem in terms of the outputs to subproblems.

# Sub-Problems

- A sub-problem is defined with respect to sub-instances.
  - Recall an instance is a specific input to the problem.
- A sub-instance is an instance restricted along some dimension.
- Example: Here is an instance of SAUL (or SASL)

$$A = [1,2,3,4,5,6,7], t = 8$$

  - Here are some sub-instances of this instance created by removing the first element one at a time:

$$A = [2,3,4,5,6,7], t = 8$$
$$A = [3,4,5,6,7], t = 8$$
$$A = [4,5,6,7], t = 8$$

  - Here are some sub-instances of this instance created by removing half of the elements at a time:

$$A = [5,6,7], t = 8$$
$$A = [7], t = 8$$

- We define a sub-problem by generalizing sub-instances to the general problem. The sub-instances above generalize to these sub-problems:

$$A[a + 1 \dots b], t$$
$$A[m + 1 \dots b], t$$

# Self-Reduction

- A reduction is a relationship between problems.
  - Formally a reduction maps inputs of one problem to inputs of another in some way that preserves output.
- A self-reduction is a reduction from a problem to its sub-problems.

- Let $U(a, b)$ express the output for the SAUL problem on input $A[a \ldots b]$ and $t$.

- Sequential Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ a & \text{if } A[a] = t \\ U(a+1,b) & \text{if } A[a] \neq t \end{cases}$$

- If we use this reduction we would get a linear search.

# Self-Reduction

- When we can find a self-reduction for a problem we say that the problem has optimal substructure.
  - Optimal substructure
- If a problem has optimal substructure we can develop a divide and conquer algorithm for it.
- There is often more than one way to reduce a problem to itself.
  - Here is another reduction that also yields a linear search.
- Let $m = \left\lfloor \frac{a+b}{2} \right\rfloor$ be the midpoint between $a$ and $b$.

Recursive Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ m & \text{if } A[m] = t \\ \max\big(U(a, m-1), U(m+1, b)\big) & \text{if } A[m] \neq t \end{cases}$$

# Self-Reduction

- Let's consider the case where the input list is sorted.
  - Both of the reductions for SAUL will still work in this case, but we can improve the second one to get a more efficient solution.

- Let $S(a, b)$ express the output for the SASL problem on input $A[a \ldots b]$ and $t$.
- Let $m = \left\lfloor \frac{a+b}{2} \right\rfloor$ be the midpoint between $a$ and $b$.

- Binary Search Reduction

$$S(a,b) = \begin{cases} -1 & \text{if } a > b \\ m & \text{if } A[m] = t \\ S(m+1, b) & \text{if } A[m] < t \\ S(a, m-1) & \text{if } A[m] > t \end{cases}$$

- If we use this reduction we would get a logarithmic search on the sorted data.

# Self-Reduction and Recursion

- We notice that the structure of a self-reduction is identical to the structure of the recursive algorithm which in turn is identical to the structure of a mathematical function expressing the runtime.

- Sequential Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ a & \text{if } A[a] = t \\ U(a+1,b) & \text{if } A[a] \neq t \end{cases}$$

```
SequentialSearch(A[a ... b], t)
        If a > b return −1.
        If A[a] = t return a.
        If A[a] ≠ t return SequentialSearch(A[a + 1 ... b], t).
End SequentialSearch
```

- Let $T(n)$ express the runtime of the sequential search algorithm on an input list of size $n$.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

# Self-Reduction and Recursion

- Recursive Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ m & \text{if } A[m] = t \\ \max\big(U(a,m-1), U(m+1,b)\big) & \text{if } A[m] \neq t \end{cases}$$

```
RecursiveSearch(A[a ... b], t)
        If a > b return −1.
        Let m = ⌊(a+b)/2⌋.
        If A[m] = t return m.
        Let c = RecursiveSearch(A[m + 1 ... b], t).
        Let d = RecursiveSearch(A[a ... m − 1], t).
        Return max(c, d).
End RecursiveSearch
```

- Let $T(n)$ express the runtime of the recursive search algorithm on an input list of size $n$.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

# Self-Reduction and Recursion

- Binary Search Reduction

$$S(a,b) = \begin{cases} -1 & \text{if } a > b \\ m & \text{if } A[m] = t \\ S(m+1,b) & \text{if } A[m] < t \\ S(a,m-1) & \text{if } A[m] > t \end{cases}$$

```
BinarySearch(A[a ... b], t)
        If a > b return −1.
        Let m = ⌊(a+b)/2⌋.
        If A[m] = t return m.
        If A[m] < t return BinarySearch(A[m + 1 ... b], t).
        If A[m] > t return BinarySearch(A[a ... m − 1], t).
End BinarySearch
```

- Let $T(n)$ express the runtime of the binary search algorithm on an input list of size $n$.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

# Recursive Analysis

- Let $T(n)$ express the runtime of the binary search algorithm on an input list of size $n$.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

- This recursively defined mathematical function is a recurrence.
  - We use recurrences to express the runtimes of recursive algorithms.

- Finding tight bounds for recurrences takes two steps:
  1. Guess a bound – recursion trees or algebraic iteration.
  2. Prove the bound – induction.

# TCSS 343 – Design and Analysis of Algorithms

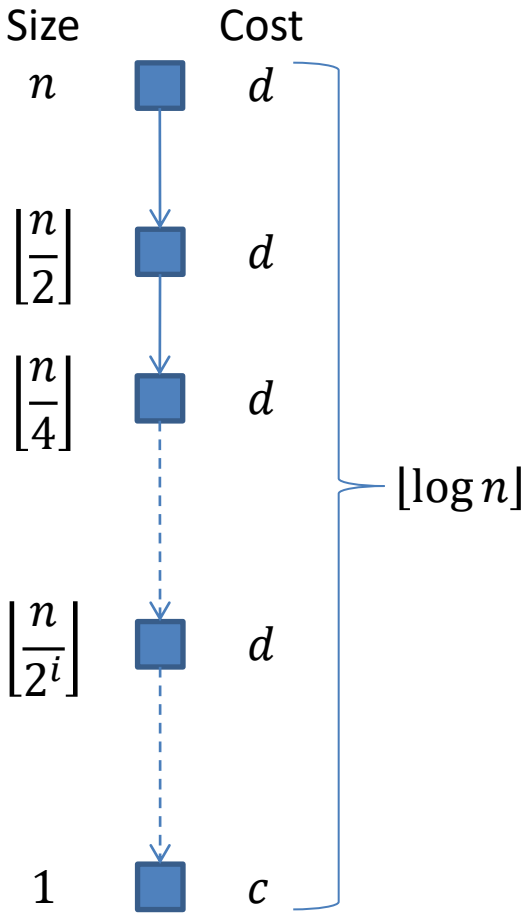
Recursion Trees

# Recursion Trees

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

- A popular method of recursive analysis is to consider the shape of the recursion tree created by the recursive call structure.
- In decrease and conquer algorithms where we have only one sub-problem the shape of the tree is a single branch.
  - Each call makes one recursive call eventually terminating at a call that is non-recursive.
- The cost of the entire algorithm is then sum of the cost of each level of this recursion.
  - This means we can rewrite the cost using summations instead of recurrences.

$$c + \sum_{i=1}^{\lfloor \log n \rfloor} d = c + d\lfloor \log n \rfloor \in \Theta(\log n)$$



# Recursion Trees

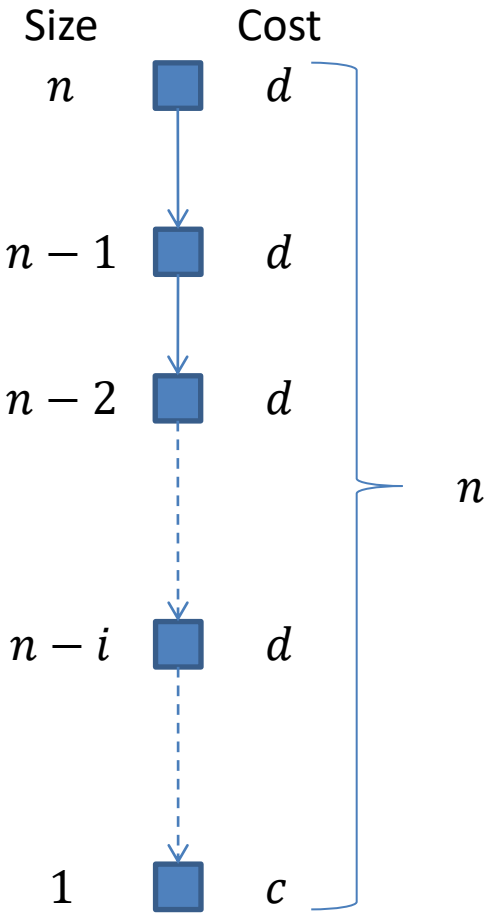
$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

- This recurrence corresponds to the linear search reduction.
- Sequential Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ a & \text{if } A[a] = t \\ U(a+1,b) & \text{if } A[a] \neq t \end{cases}$$

- This is another decrease and conquer algorithm where we have only one sub-problem.

$$c + \sum_{i=1}^{n-1} d = c + dn - d \in \Theta(n)$$



# Recursion Trees

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

- This recurrence corresponds to the second linear search reduction.
- Recursive Search Reduction

$$U(a,b) = \begin{cases} -1 & \text{if } a > b \\ m & \text{if } A[m] = t \\ \max\big(U(a, m-1), U(m+1, b)\big) & \text{if } A[m] \neq t \end{cases}$$

- This is not a decrease and conquer algorithm since we have two sub-problems.
- The shape of this tree is a complete binary tree of depth $k = \lfloor \log n \rfloor$.

$$c2^{\lfloor \log n \rfloor} + \sum_{i=1}^{\lfloor \log n \rfloor - 1} d2^i$$

$$= c2^{\lfloor \log n \rfloor} + d \sum_{i=1}^{\lfloor \log n \rfloor - 1} 2^i$$

$$= c2^{\lfloor \log n \rfloor} + d2^{\lfloor \log n \rfloor} - d$$

$$\leq cn + dn - d \in \Theta(n)$$

